**2026 Fall — Graduate Operating Systems**

Mid-term Progress Report (Improvement Paper)

# PeakBaleen: Peak-Aware Flash-Cache Admission via Ising-Machine Episode Selection

*An improvement on Baleen: ML Admission & Prefetching for Flash Caches (USENIX FAST '24)*

**Prepared by**

Ching-Hao Chiu · Jie Fu · Sixue Xing

University of Notre Dame, Notre Dame

## 1. Summary of Progress

Our proposal asked two questions. Can selecting cache admissions by a joint min–max criterion, solved with an Ising solver, lower peak disk load at the same flash-write budget? And does that gain survive deployment through Baleen's online model? We have reproduced the baseline and built the full pipeline. The baselines and the load-adaptive threshold have been evaluated on five of the seven Meta traces, and the full design on three. A sixth trace, Region2, cannot be fully evaluated at this write budget. Three results stand so far.

- **Offline, the idea works.** Peak-aware selection lowers the peak by 13.7% against Baleen's own offline oracle and wins on 37 of 39 samples.
- **Deployed, the full design does not work yet.** It failed its pre-registered test, landing 3.2% above Baleen on Regions 6 and 7, and it came out 5.3% above on Region5, where it was reported without a pass rule. Labels learned on the training day break when a test-day peak is of a kind that day did not contain.
- **One component works on its own.** A load-adaptive admission threshold, added to Baleen's own models, passed a pre-registered test on Region5 (−3.2%). It also lowers the peak on Regions 4, 6 and 7, observed after the fact, and does not help on the low-load Region1.

The Ising solver ties the best classical solvers on the offline problem, so the offline gain comes from the formulation and not from the solver. Table 1 sets these results against the proposal's hypotheses.

*Table 1. Status of the proposal's hypotheses.*

| Hypothesis (proposal §3.1) | Status | Evidence so far |
|---|---|---|
| H1. Min–max selection lowers Peak DT by at least 5% relative to stock Baleen, averaged over the seven traces | Partly met | Offline, against Baleen's own oracle: 13.7% on Regions 4–7 (Region4 alone: 5.4%); Regions 1–3 not yet run offline. Deployed: not met (Table 3). |
| H2. An Ising solver on a sum-of-squares surrogate beats the artifact's greedy peak policy and a greedy warm start by at least 3% | Not met as stated | The surrogate was the weakest Ising route, so we moved to a smoothed-peak energy (§4.2). On that objective every solver beats peak-aware greedy by 13% or more, LP included, so the margin does not distinguish Ising. The artifact's own peak policy has not been run yet. |
| H3. An online model on peak-aware labels retains at least 60% of the offline gain | Not met | Retention (the share of the offline gain kept online) of the full design is negative: −0.28 on Regions 6 and 7, −0.54 on Region5. The adaptive bar alone retains +0.16 and +0.31. |

## 2. The Task

Read requests hit a flash cache in front of hard disks. A hit costs the disks nothing; a miss costs disk-head time. On each miss, an admission policy decides whether to write the block into flash. Flash wears out as it is written, so the long-term write rate is capped. Our problem is therefore to decide what enters the cache, under a fixed write budget, so that the peak disk load is as low as possible.

**Metric and budget.** We minimize P100, the busiest 10-minute window of backend disk utilization after the first day, because disks are provisioned for that window. Every comparison targets a flash write rate of 35.6 MB/s within ±1%, with a 366 GB cache. The proposal quoted 400 GB and three drive-writes per day; the authors' Figure 9 runs use the values above, so we adopted them to stay comparable with the paper.

**Data.** We use Baleen's public traces from Meta's Tectonic storage system [1, 2]. Each trace comes from one storage cluster. The paper labels the seven clusters Region1 to Region7; the name is anonymous and not geographic. Regions 1–3 were recorded in 2019, Region4 in 2021 and Regions 5–7 in 2023. Each trace is downsampled to 0.1% of its blocks, keeping every access to those blocks, and the cache is scaled to match. Samples s0, s0.1 up to s0.9 are ten different slices of the same cluster, so each is a separate smaller workload. The first day of a trace is used for training and the remaining days for testing. Everything runs in BCacheSim, Baleen's own simulator, which we keep frozen.

**How we test.** We tune on development samples (s0–s0.3 of Regions 6 and 7) and test on other samples. For a pre-registered test, we commit the hypothesis and the pass rule to git before running it. Samples s0.7–s0.9 are held back: no deployed run has used them, and they will be run once at the end. Baleen's retraining is not deterministic, so each configuration in a test is retrained at least three times.

## 3. Baselines We Ran

Table 2 gives the baselines we ran ourselves. CoinFlip admits at random, RejectX rejects first-time items, and Baleen is the machine-learned policy. The last column is Baleen's OPT, the offline oracle that Baleen's model is trained to imitate.

*Table 2. Peak disk load P100 (%), lower is better, all at 35.6 MB/s.*

| Trace | Year | Samples | CoinFlip | RejectX | Baleen | Baleen's OPT |
|---|---|---|---|---|---|---|
| Paper, Fig. 9 | 2019–23 | avg of 7 | 34.63 | 31.95 | 28.11 | — |
| Region7 | 2023 | s0 | 48.96 | 42.46 | 40.10 | 34.52 |
| Region6 | 2023 | s0 | 43.45 | 42.32 | 43.25 | 35.56 |
| Region6 + 7 | 2023 | s0.4–0.6 | 48.64 | 40.73 | 39.27 | 33.62 |
| Region5 | 2023 | s0.1–0.6 | 44.47 | 39.35* | 36.17 | 30.10 |
| Region4 | 2021 | s0.2–0.6 | 68.84 | 65.84 | 50.72 | 38.13 |
| Region1 | 2019 | s0.1–0.6 | 50.39 | 28.20 | 24.14 | not run |

The first row is the paper's Figure 9 average, recomputed from the authors' results file. s0 rows are our reproduction; later samples are held-out test sets. Baleen is the mean of 3–9 retrains. *RejectX cannot reach the write rate on 5 of the 6 Region5 samples, because its write rate jumps across the target between adjacent settings.

Baleen is the strongest deployed baseline on every set except Region6 s0, where RejectX is lower. Two facts set the tone for our method. On the held-out sets, Baleen online sits 14–25% above its own offline oracle

(14.4% on Regions 6 and 7, 16.8% on Region5, 24.8% on Region4), so there is headroom. And that oracle optimizes the average, not the peak.

## 4. Our Design

### 4.1 Three components

PeakBaleen keeps Baleen's pipeline and its online datapath, and no solver runs at request time. It changes three things, which can be switched on separately (Figure 1).

- **Component 1: new labels.** Offline, on the training day, we replace Baleen's ranking of episodes with a selection that minimizes the busiest window under the same write budget, solved by an Ising solver. We call the solution our peak-aware oracle. It becomes the training labels for Baleen's gradient-boosted model (GBM).
- **Component 2: a load feature.** The GBM also sees how busy the disks were: the disk time of all read requests in the last 10, 30 and 60 minutes. The signal uses only past requests and comes from the request stream, before the cache, so it is identical in training and serving.
- **Component 3: an adaptive bar.** The GBM scores each request, and the request is admitted if its score exceeds a threshold θ. Baleen's threshold is fixed. Ours follows the load of the last 10 minutes, falling when the disks are busy and rising when they are quiet:

$$\theta_t = \theta \cdot \mathrm{clip}\left( (\mathrm{load}_t / 20)^{-1}, 0.25, 4 \right)$$

The base θ is still tuned so that the write rate meets the same budget, so the bar moves flash writes toward busy periods and does not add any.

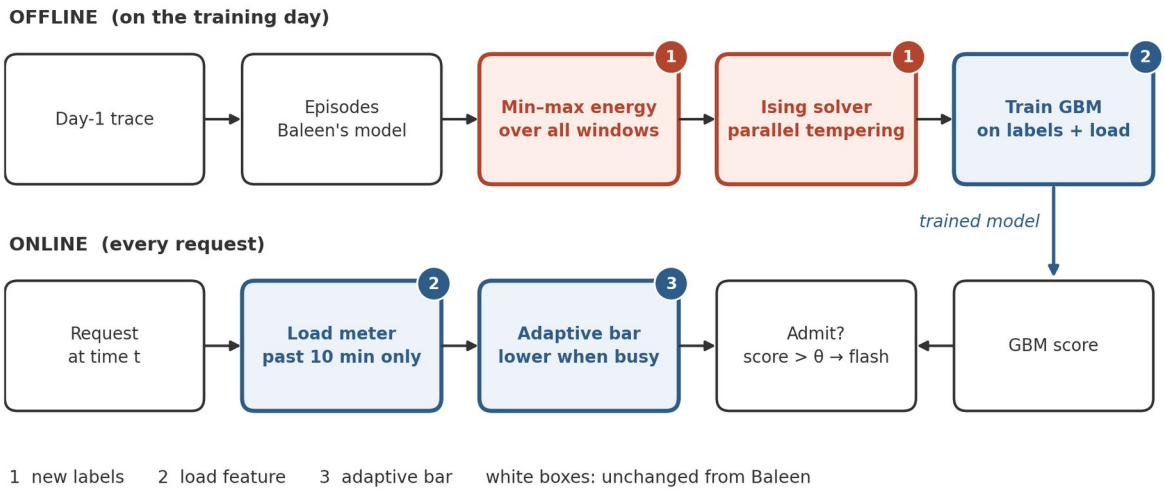



*Figure 1. PeakBaleen's design. Colored boxes are new; numbers mark the three components.*

### 4.2 Algorithm

Algorithm 1 is Baleen as released. Algorithm 2 is PeakBaleen, with the same steps in gray, the Ising core in orange, and the other new steps in blue.

**Algorithm 1** Baleen (baseline)

**Require:** episodes $\mathscr{E}$ with size $s_e$ and disk time saved $d_e$; flash write budget $B$

1: sort $\mathscr{E}$ by $d_e/s_e$, descending ▷ average-optimal OPT
2: $y_e \leftarrow 1$ on the longest prefix whose writes fit in $B$, else 0
3: $f \leftarrow \textsc{TrainGBM}(\phi, y)$
4: **for** each access $a$ **do**
5: admit $a$ **iff** $f(\phi(a)) > \theta$
6: **end for**
7: tune $\theta$ so the flash write rate = $B$

*Algorithm 1. Baleen: rank episodes by disk time saved per byte, label the prefix that fits the budget, imitate it online.*

**Algorithm 2** PeakBaleen (ours)

**Require:** as Alg. 1, plus per-window disk time saved $d_{e,w}$ and no-cache load $C_w$

1: $x \leftarrow$ LP relaxation, rounded and repaired ▷ warm start
2: $L_w(x) \leftarrow C_w - \sum_e d_{e,w}\, x_e$ ▷ load left in window $w$
3: $E(x) \leftarrow \mathrm{LSE}_\gamma(L(x)) + P\,(\sum_e s_e x_e - B)^2$ ▷ min–max QUBO
4: $x \leftarrow \textsc{ParallelTempering}(E,\ x,\ 16 \text{ replicas})$ ▷ Ising solver
5: $y_e \leftarrow x_e$ ▷ peak-aware labels
6: $f \leftarrow \textsc{TrainGBM}([\phi, \ell], y)$
7: **for** each access $a$ at time $t$ **do**
8: $\ell_t \leftarrow$ GET disk time in $[t - 10\,\text{min},\ t)$ ▷ causal load
9: $g \leftarrow \mathrm{clip}((\ell_t/20)^{-1},\ 0.25,\ 4)$
10: admit $a$ **iff** $f(\phi(a), \ell_t) > \theta \cdot g$
11: **end for**
12: tune $\theta$ so the flash write rate = $B$

*Algorithm 2. PeakBaleen: changes to Baleen, line by line.*

**Formulation and solver.** The proposal planned a pure QUBO, a sum-of-squares penalty on window loads solved by simulated bifurcation [5]. We implemented it, and it was the weakest of our four Ising solvers in development tuning. The finalist instead runs parallel tempering [4] (replica exchange with 16 replicas) on the energy in line 3, started from a classical LP solution. Strictly, only the budget penalty in that energy is quadratic; the peak term is a log-sum-exp smooth maximum, so we use "QUBO" loosely in the algorithm. The solver runs on CPU, and no Ising hardware is involved, as the proposal anticipated.

## 5. Results So Far

We compared PeakBaleen with Baleen from three angles: the offline labels alone, the full design deployed, and the adaptive bar alone deployed. Deployed means replaying the test days in the simulator, with the trained model deciding request by request without seeing the future. Table 3 gives the results, and Figure 2 shows the samples on which both deployed designs ran.

*Table 3. Peak disk load P100 (%), lower is better.*

| Setting | Baleen | Ours | Change | Evidence |
|---|---|---|---|---|
| Offline: our oracle vs Baleen's OPT, Regions 4–7 | 33.00 | 28.59 | −13.7% | Wins 37 of 39 samples |
| Full PeakBaleen, Region6 + 7 | 39.27 | 40.53 | +3.2% | Pre-registered: fails; wins 5 of 6, one collapse |
| Full PeakBaleen, Region5 | 36.17 | 38.09 | +5.3% | Reported arm; wins 3 of 6, one collapse |
| Adaptive bar only, Region6 + 7 | 39.27 | 38.30 | −2.5% | Observed (control arm); wins 5 of 6 |
| Adaptive bar only, Region5 | 36.17 | 35.02 | −3.2% | Pre-registered: passes; wins 5 of 6 |
| Adaptive bar only, Region4 | 50.72 | 42.87 | −15.5% | Observed, not pre-registered; wins 5 of 5 |
| Adaptive bar only, Region1 (low load) | 24.14 | 24.62 | +2.0% | Observed, not pre-registered; wins 1 of 6 |

In the first row the Baleen column is Baleen's offline OPT, averaged over 39 samples. All other rows compare deployed policies with deployed Baleen, using the Baleen numbers from Table 2. Wins count samples below Baleen. The shaded row is the one pre-registered positive result.

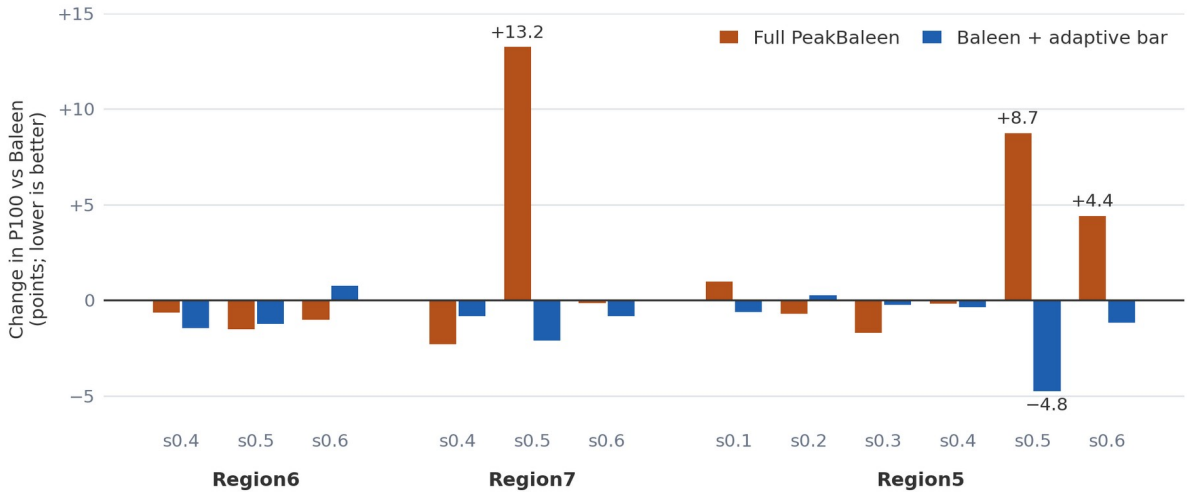



*Figure 2. Change in peak load against Baleen on each deployed test sample of Regions 5, 6 and 7.*

**Offline.** Our oracle wins all 30 samples on Regions 5–7, by about 16% on average. Region4 is the exception, at 7 of 9 wins and 5.4%: its peaks are short bursts that Baleen's oracle already absorbs as far as the budget allows. The peak does migrate: the busiest simulated window differs from the Baleen oracle's on 33 of 39 samples. The min–max objective still lowers it.

**Full design.** Full PeakBaleen beats Baleen on five of six samples in its pre-registered test, but Region7 s0.5 rises from 39.07 to 52.30, which turns the mean negative. The same happens on Region5 s0.5 (42.72 to 51.46), and Region5 s0.6 also rises, by 4.40 points.

**Adaptive bar alone.** This is the one positive deployed result, and it has limits. It passed its pre-registered rule on Region5, a cluster we had never run deployed, but 69% of the gain comes from one sample, s0.5. The mean gain is 1.16 points with an instance-level standard error of 0.75; the pass rule uses the retrain-level

standard error, 0.09. On Region4 the gain is large, but we found it after the fact, and two samples carry 93.5% of it. On Region1, whose mean load is much lower, it does not help. That suggests the benefit depends on load, which we treat as a hypothesis to test and not as a finding.

**Earlier load-based attempts.** The Baleen authors tried admitting only during high load, and it saved flash writes but did not reduce the peak [1, §5.6]. Wong's thesis reports load-dependent thresholds on OPT cutting flash writes by 8.0% and the peak by only 1.4% on Region3 [3, §7.4.1]. Our bar scales the trained model's threshold continuously and re-tunes the base θ to the same write rate. Whether these differences explain our different outcome has not been measured.

**What each component adds.** Table 4 separates the components. The adaptive bar carries the deployed gain. The load feature alone does not help and hurts on Region4. Adding the new labels on top of the other two makes the peak worse. Among the positive cells, only the adaptive bar on Region5 is a pre-registered claim.

*Table 4. Deployed P100 (%) by component, with change against Baleen.*

| Configuration | Region6 + 7 | Region5 | Region4 | Region1 |
|---|---|---|---|---|
| Baleen (reference) | 39.27 | 36.17 | 50.72 | 24.14 |
| + load feature (2) | 39.17 (−0.2%) | 36.37 (+0.5%) | 56.71 (+11.8%) | 24.54 (+1.7%) |
| + adaptive bar (3) | 38.30 (−2.5%) | 35.02 (−3.2%) | 42.87 (−15.5%) | 24.62 (+2.0%) |
| + load feature + adaptive bar (2, 3) | 38.38 (−2.3%) | 34.48 (−4.7%) | 43.87 (−13.5%) | 24.48 (+1.4%) |
| Full PeakBaleen (1, 2, 3) | 40.53 (+3.2%) | 38.09 (+5.3%) | not run | not run |

## 5.1 Why the labels do not transfer

Some peak patterns appear on test days but not at the peak of the training day. The one that hurts us is a big sequential scan at the busiest time: many requests reading large blocks in order, far into each block. In Region7 s0.5's peak window, 48% of the accesses end beyond 8 MB. Region7's training-day peaks are not scans, so our oracle has no reason to select that data, and the model learns that big-offset episodes are not worth caching. On the test day it admits only 4–14% of that window, where Baleen's models admit 56–69%. The scan goes to the disks and the peak rises by about 13 points. Region5 s0.5 fails the same way: its peak window is 52% big-offset accesses, and admission there is 0.28 against 0.39 for Baleen and 0.56 with the adaptive bar.

The same labels help when the training day does contain the pattern. Region6's peaks are scans, and in development runs the full design admits 41% of the peak window against Baleen's 22%, moving the peak off the scan in 10 of 12 retrains. So whether the training-day peak contains the pattern decides whether the labels help or hurt.

## 5.2 Solver check

We solved the same offline problem with LP and with CP-SAT. They lower the peak by 13.0% and 13.2%, against 13.7% for the Ising solver, and the means differ by less than 0.2 points. Sample by sample, the Ising solver is lower than CP-SAT on 18 of 39 and lower than LP on 22 of 39, so we read this as a tie. A separate pre-registered test at tight budgets of 1, 3 and 10 seconds also failed: parallel tempering matched or beat the best classical method on 1, 6 and 4 of 6 samples, and the rule required at least 5 of 6 at two budgets or more. The gain comes from the min–max formulation.

## 6. Challenges Identified and Solved

**A metric that changed after the paper.** Our first reproduction missed the paper's Figure 9 Baleen bars for Regions 6 and 7 (37.59 and 41.31; we got 40.10 and 43.25). We traced this to an older, unreleased version of the disk-time metric used in those runs. Measured with the released metric, RejectX and CoinFlip match the authors' runs bit-exactly and Baleen matches within 0.6 points. Running the authors' own notebook on their results file gives the paper's averages exactly (28.11, 31.95, 34.63).

**Keeping the write budget honest.** A peak gain bought with extra flash writes is not a gain, so the threshold of every Baleen-based policy is converged by full simulation to 35.6 MB/s within ±1%. Runs that miss are excluded from means and reported. This caught a bug of our own: our threshold search stopped at 1, while the adaptive bar can need a higher base value, which left 36 development rows unmatched until we widened it.

**A load signal that cannot cheat.** In our first phase, peak-aware labels did not survive into Baleen's model, because the model has no signal for time or load. The proposal planned trailing backend load, but that depends on the admission policy, which risks a mismatch between training and serving. We use demand-side load instead, taken from the request stream with a strictly past window. A parity test found exact equality between training and serving on 26,499 accesses.

**Changing the simulator without disturbing the baseline.** All changes live in an overlay patch (9 files, +217/−6 lines) on a frozen copy of the artifact, with every new feature off by default. With defaults, the overlay reproduces the frozen artifact's labels and model files byte for byte, and results are identical. The teammate review of this patch that we had planned was waived before the test runs; these checks stand in its place.

**Problem size.** The proposal worried about instances beyond a dense QUBO. Real instances have 32k–89k episodes over about 1,000 windows. Our matrix-free simulated bifurcation stays within 214 MB at 126k variables, where the dense package runs out of memory from about 70k variables on our 123 GB machine. Parallel tempering works on the sparse form and stays within 165 MB.

**Still open: making labels transfer.** Three attempts have not solved this. First, deployment-aware objectives (anchored to Baleen's labels, robust top-k, student designs): none passed the development screen over 64 full simulations, because anchoring repairs Region7 but gives back Region6's scan gain. Second, an Ising-fitted threshold table by load level: it failed its pre-registered test on Region1 (24.45 against 24.14). Third, an unattended agent search (a Claude Opus 5.5 worker with independent validator agents, under a written contract allowing two test runs). Its best design, a union of Baleen's model and a peak-label model, beat the adaptive bar in development (35.36 against 36.16), but on Region4 it landed at 43.17 against the bar's 42.87. Its other test, on Region2, could not be evaluated: that trace peaks near 10% and sample s0.2 reaches only 11.5 MB/s even when admitting everything.

## 7. What We Will Finish for the Final Paper

- **Finish the baselines.** Add Region3, on the samples the authors' release covers, and report Region2 where it can reach the budget.
- **Make the oracle's labels transfer.** Next we will train over several days, so that the labels cover more kinds of peak. This widens the day-1 training window we have kept fixed so far. If the labels still do not transfer, the oracle-versus-online decomposition planned in proposal §4.5 becomes the main finding.
- **Ablations.** Extend Table 4, test when load-awareness helps (load level, peak type), and run the artifact's own peak policy, which the proposal's H2 names and we have not yet measured.

- **Final test and paper.** One run on samples 0.7–0.9, with the configurations and pass rules committed first, and a conference-format paper.

*Table 5. The proposal's timeline and where we stand.*

| Dates (2026) | Phase | Deliverable / exit criterion | Status |
|---|---|---|---|
| Sep 10 – 20 | Literature review | Venue survey; Baleen chosen; artifact audited (builds, data sized) | Done |
| Sep 21 – Oct 5 | Baseline & problem definition | Baleen reproduced on 7 traces; Fig. 9 peak-load numbers matched; per-window DT extractor; QUBO spec frozen | Done, with two exceptions: 5 of 7 traces so far; the paper's Region6/7 Baleen bars cannot be reproduced (§6) |
| Oct 6 – 20 | Algorithm design | Min–max surrogate + reweighting; QUBO builder; solver comparison (SB / SA / ILP) on pruned instances | Done; formulation and solver changed (§4.2) |
| Oct 21 – Nov 1 | Implementation & analysis | PeakBaleen policy in BCacheSim; offline-oracle Peak DT measured vs Baleen; online GBM trained on peak-aware labels | Done; offline positive, online negative (§5) |
| Nov 2 – 15 | Full evaluation | All 7 traces × 3 samples; ablations; per-trace results table; attribution analysis | In progress: five traces evaluated, first ablation table in hand (Table 4) |
| Nov 16 – Dec 2 | Write-up & final talk | Final paper (conference format) + presentation; artifact and scripts released | Not started; includes the final test on samples 0.7–0.9 |

Both of the proposal's hard gates are met: the baseline is reproduced and we have an offline result.

## References


[1] Wong, Daniel Lin-Kit, et al. "Baleen: ML admission & prefetching for flash caches." 22nd USENIX Conference on File and Storage Technologies (FAST 24). 2024.

[2] Pan, Satadru, et al. "Facebook's Tectonic filesystem: Efficiency from exascale." 19th USENIX Conference on File and Storage Technologies (FAST 21). 2021.

[3] Wong, Daniel Lin-Kit. "Machine learning for flash caching in bulk storage systems." PhD thesis, Carnegie Mellon University, CMU-CS-24-152. 2024.

[4] Earl, David J., and Michael W. Deem. "Parallel tempering: Theory, applications, and new perspectives." Physical Chemistry Chemical Physics 7.23 (2005): 3910–3916.

[5] Goto, Hayato, Kosuke Tatsumura, and Alexander R. Dixon. "Combinatorial optimization by simulating adiabatic bifurcations in nonlinear Hamiltonian systems." Science Advances 5.4 (2019): eaav2372.


**Code and results.** https://github.com/xingsixue123/OS_final_project (branch phase2-deployable-peak). Every number in this report is traced to its source file in changes/5_midterm/SOURCES.md.